

Reddit-Style Notifications System – Summary

■ Goal

Build a real-time, multi-tenant notification platform with:

- Sub-ms fanout in-region
- Single-digit ms read latency
- High reliability, scalability, cost-awareness, and compliance

■ Key Outcomes

- Delivery: <1s to online users, a few secs for push/email
- Correctness: no duplicates, ordered per-user, at-least-once inbox delivery
- Controls: user preferences, quiet hours, spam/abuse filtering, rate limiting
- Multi-tenancy: quotas, budgets, isolation
- Operational excellence: clear SLOs, observability, rollback

■ Assumptions

- Tenants: 1 → 1000s
- Users: millions → 100M+
- Peak write rate: 100K–1M events/sec
- Read QPS: 50K–500K
- Events: replies, mentions, votes, awards, chat, mod actions, system messages

■ Functional Requirements

- Event capture from posts, comments, votes, mod tools, campaigns
- Fanout notifications to multiple channels with ordering + de-duplication
- Channels: in-app inbox, websockets, push, email
- Features: preferences, unread counts, quiet hours, spam detection
- Admin: replay, suppression, backfills, campaigns

■ Non-Functional Requirements

- Availability: 99.95%
- Latency: p95 websocket <100ms, inbox fetch <10ms
- Durability: RPO=0
- Compliance: SOC2, GDPR, CCPA

■ Architecture

Producers → Ingestion (API Gateway, Kafka) → Stream Processing (Flink/Kafka Streams)
→ Redis hot inbox → DynamoDB/Scylla durable store → Websockets/Push/Email

■ Storage

- Hot path: Redis (lists, sets, counters)
- Durable path: DynamoDB/Scylla
- Search: OpenSearch
- Analytics: S3/Parquet → ClickHouse/BigQuery

■ APIs

- POST /v1/events → publish event
- GET /v1/inbox → recent notifications
- GET /v1/counters → unread counters
- POST /v1/ack → mark read/unread/hide
- GET /v1/preferences → fetch preferences
- PATCH /v1/preferences → update preferences
- Websocket → subscribe for real-time stream